Design and Verification Report: 16-bit Combinatorial ALU

Digital Logic Design Documentation

December 20, 2025

1 Introduction

The Arithmetic Logic Unit (ALU) is a fundamental component of a Central Processing Unit (CPU) responsible for performing bitwise logical and arithmetic operations. This report details a **16-bit ALU** implemented in VHDL, supporting four core operations and providing a `Zero` status flag.

2 Design Flow

The ALU was developed using a standard RTL design methodology:

1. **Functional Specification:** Defining a 2-bit opcode (`Op`) to select between logical (AND, OR) and arithmetic (ADD, SUB) functions.
2. **RTL Modeling:** Implementing the logic using a combinatorial process with a `case` statement for operation selection.
3. **Status Logic Design:** Integrating a concurrent equality check to drive the `Zero` flag.
4. **Functional Verification:** Utilizing a behavioral testbench to apply stimulus vectors and observe output responses in a waveform viewer.

3 Functional Description

The ALU is primarily combinatorial. While it includes a `CLR` signal, the operations respond immediately to changes in the input operands `A`, `B`, or the opcode `Op`.

3.1 Opcode Mapping

The 2-bit `Op` signal determines the operation as follows:

Opcode	Operation	Description
00	AND	Bitwise logical AND of A and B
01	OR	Bitwise logical OR of A and B
10	ADD	16-bit Unsigned Addition
11	SUB	16-bit Unsigned Subtraction

Table 1: *ALU Operation Table*

3.2 Zero Flag Generation

The `Zero` flag is implemented as a concurrent assignment. It monitors the internal signal `Y` (the result) and asserts '1' if and only if all 16 bits of the result are zero.

$$Zero = \begin{cases} 1 & \text{if } Result = 0\text{x}0000 \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

4 Key Technical Highlights of the Design

4.1 Combinatorial Efficiency

Unlike sequential components, this ALU is designed for high-speed operation by avoiding clock dependency. The result is calculated in parallel across all functional units, with the final value selected by an **RTL Multiplexer** structure synthesized from the `case` statement.

4.2 Arithmetic Type Casting

The design correctly utilizes the `IEEE.NUMERIC_STD` library. Input `std_logic_vector` operands are cast to `unsigned` types for addition and subtraction to ensure correct hardware mapping of carry-chains, before being cast back for the output port.

4.3 Optimized Zero Detection

The `Zero` flag uses an equality operator rather than a ROM lookup or a sequential loop. This results in the synthesis of a **wide NOR gate tree**, providing the lowest possible propagation delay ($O(\log n)$) for flag generation.

5 Testbench and Verification

The testbench (`tb_ALU`) validates the data path by cycling through each operation with known constants.

- **Logical Tests:** Verifying bitwise masks (e.g., `0x0F0F` AND `0x00FF`).
- **Arithmetic Tests:** Confirming sum and difference accuracy.
- **Flag Validation:** Specifically testing $1 - 1 = 0$ to ensure the `Zero` flag is correctly triggered.
- **Asynchronous Clear:** Verifying that the `CLR` signal overrides current calculations and forces the result to zero.

6 Conclusion

The 16-bit ALU design provides a robust foundation for a Mini-RISC processor. It demonstrates proper VHDL coding standards, efficient use of numeric libraries, and a verification-first approach to hardware development.